

Treasure Map

Binary Search Trees, Validation & Search · Mentor-Led Lab (Student Edition)

Developed by TalenciaGlobal

Difficulty ★★★ Advanced

Problem

Validate that a tree is a binary search tree, then search it for a key. You are given starter code with two functions. `is_bst` is almost complete but has a bug, and `search` is an empty stub. Fix the bug and implement the stub.

What the Program Reports

- Valid BST: Yes when the tree obeys the BST property, otherwise No.
- Found <key>: Yes if the key appears anywhere in the tree, otherwise No.

Input / Output Format

Input: Line 1: the tree in level order (values and N). Line 2: the search key.

Output: Two lines: Valid BST and Found <key>.

Examples

Example 1

Input: tree = 5 3 8 2 6, key = 6

Output:

Valid BST: No
Found 6: Yes

Explanation: 6 sits in the left subtree of 5 yet exceeds 5, so the tree is not a valid BST; 6 is still present, so the search finds it.

Example 2

Input: tree = 5 3 8 2 4 7 9, key = 4

Output:

Valid BST: Yes
Found 4: Yes

Explanation: A proper BST containing 4.

Constraints

- A valid BST requires every left-subtree value below a node and every right-subtree value above it.
- The search scans the whole tree, so it works whether or not the tree is a BST.
- Values are integers.

Your Task

- Task 1 - Fix the bug. `is_bst` is flagged with a comment, but the bug's location is not given. Find the single bug so validity is judged against inherited (lo, hi) bounds, not just the immediate children.
- Task 2 - Implement search from scratch: return whether the key appears anywhere in the tree.

Starter Code

Begin from the code below. One function carries a single bug (flagged by a comment, with no hint to its location); the other is an empty stub you must implement.

Python

```
# Lab T4 - Treasure Map (validate a BST, then search the tree for a key)
import sys
from collections import deque
class Node:
    def __init__(self, v): self.v = v; self.left = None; self.right = None
def build(tokens):
    if not tokens or tokens[0] == "N": return None
    root = Node(int(tokens[0])); q = deque([root]); i = 1
    while q and i < len(tokens):
        node = q.popleft()
        if i < len(tokens):
            if tokens[i] != "N": node.left = Node(int(tokens[i])); q.append(node.left)
            i += 1
        if i < len(tokens):
            if tokens[i] != "N": node.right = Node(int(tokens[i])); q.append(node.right)
            i += 1
    return root
# This function contains a BUG. Find it and fix it.
# A valid BST requires EVERY node in the left subtree to be smaller and every
# node in the right subtree to be larger - not just the immediate children.
def is_bst(n, lo, hi):
    if not n: return True
    if n.left and n.left.v >= n.v: return False
    if n.right and n.right.v <= n.v: return False
    return is_bst(n.left, lo, n.v) and is_bst(n.right, n.v, hi)
# TODO: Implement this function from scratch.
# Return True if key appears anywhere in the tree, otherwise False.
def search(n, key):
    return False
```

```
def main():
    lines = sys.stdin.read().splitlines()
    root = build(lines[0].split()); key = int(lines[1])
    print("Valid BST: " + ("Yes" if is_bst(root, float("-inf"), float("inf")) else "No"))
    print("Found {}: {}".format(key, "Yes" if search(root, key) else "No"))
main()
```

C++

```
#include <iostream>
#include <vector>
#include <string>
#include <queue>
#include <sstream>
#include <functional>
#include <algorithm>
using namespace std;
struct Node { long long v; Node* left; Node* right; Node(long long
x):v(x),left(nullptr),right(nullptr){ } };
Node* build(const vector<string>& t){
    if (t.empty() || t[0]=="N") return nullptr;
    Node* root=new Node(stoll(t[0])); queue<Node*> q; q.push(root); size_t i=1;
    while(!q.empty() && i<t.size()){ Node* node=q.front(); q.pop();
        if (i<t.size()){ if (t[i]!="N"){ node->left=new Node(stoll(t[i])); q.push(node->left);} i++; }
        if (i<t.size()){ if (t[i]!="N"){ node->right=new Node(stoll(t[i])); q.push(node->right);} i+
+; } }
    return root;
}
vector<string> readTokens(){ vector<string> t; string s; while(cin>>s) t.push_back(s); return t; }
#include <climits>
// This function contains a BUG. Find it and fix it.
// A valid BST requires EVERY node in the left subtree to be smaller and every
// node in the right subtree to be larger - not just the immediate children.
bool isBst(Node* n, long long lo, long long hi){ if(!n) return true; if(n->left && n->left->v>=n->v)
return false; if(n->right && n->right->v<=n->v) return false; return isBst(n->left,lo,n-
>v)&&isBst(n->right,n->v,hi); }
// TODO: Implement this function from scratch.
// Return true if key appears anywhere in the tree, otherwise false.
bool search(Node* n, long long key){ return false; }
int main(){
    string l1,l2; getline(cin,l1); getline(cin,l2);
    istringstream iss(l1); vector<string> t; string s; while(iss>>s) t.push_back(s);
    long long key=stoll(l2); Node* r=build(t);
    cout<<"Valid BST: "<<(isBst(r,LLONG_MIN,LLONG_MAX)?"Yes":"No")<<"\n";
    cout<<"Found "<<key<<": "<<(search(r,key)?"Yes":"No")<<"\n"; return 0; }
```

Java

```
import java.util.*;
public class Main {
    static class Node { long v; Node left, right; Node(long x){v=x;} }
    static Node build(String[] t){ if (t.length==0||t[0].equals("N")) return null; Node root=new
Node(Long.parseLong(t[0])); Queue<Node> q=new LinkedList<>(); q.add(root); int i=1;
    while(!q.isEmpty()&&i<t.length){ Node n=q.poll(); if(i<t.length){ if(!t[i].equals("N"))
{ n.left=new Node(Long.parseLong(t[i])); q.add(n.left);} i++; } if(i<t.length){ if(!t[i].equals("N")){
n.right=new Node(Long.parseLong(t[i])); q.add(n.right);} i++; } } return root; }
    // This function contains a BUG. Find it and fix it.
```

```
// A valid BST requires EVERY node in the left subtree to be smaller and every
// node in the right subtree to be larger - not just the immediate children.
static boolean isBst(Node n, long lo, long hi){ if(n==null) return true; if(n.left!=null &&
n.left.v>=n.v) return false; if(n.right!=null && n.right.v<=n.v) return false; return
isBst(n.left,lo,n.v)&&isBst(n.right,n.v,hi); }
// TODO: Implement this function from scratch.
// Return true if key appears anywhere in the tree, otherwise false.
static boolean search(Node n, long key){ return false; }
public static void main(String[] a){ Scanner sc=new Scanner(System.in);
String l1=sc.nextLine().trim(); long key=Long.parseLong(sc.nextLine().trim());
Node r=build(l1.split("\\s+"));
System.out.println("Valid BST: "+
(isBst(r,Long.MIN_VALUE,Long.MAX_VALUE)?"Yes":"No"));
System.out.println("Found "+key+": "+(search(r,key)?"Yes":"No")); }
}
```

Sample Run

Sample Input

```
5 3 8 2 6
6
```

Sample Output

```
Valid BST: No
Found 6: Yes
```